

Fonctions Avancées, Paramétrage et Gestion des Erreurs

1. Industrialisation des Scripts : Le Bloc de Paramètres param()

1.1 Limites des valeurs codées en dur (Hardcoding)

Dans un script débutant, les variables de travail contiennent souvent des valeurs figées :

```
$DossierCible = "C:\Windows"  
$SeuilTailleMo = 50
```

Cette approche présente deux défauts majeurs en environnement de production :

- Pour modifier la cible ou un seuil, l'administrateur doit ouvrir le code source et modifier le texte du script, ce qui expose à des modifications accidentelles.
- Le script ne peut pas être intégré dans une chaîne d'automatisation ou un ordonnanceur nécessitant le passage d'arguments variables.

1.2 Déclaration et positionnement du bloc param()

Le bloc param() permet d'exposer des paramètres utilisables directement depuis la ligne de commande lors de l'appel du script :

```
.\audit_securise.ps1 -DossierCible "D:\Donnees" -SeuilTailleMo 100
```

Règle syntaxique impérative : Le bloc param() doit obligatoirement être la toute première instruction exécutable du fichier (seuls des commentaires d'en-tête peuvent le précéder).

1.3 Typage fort des données

Déclarer le type attendu devant chaque variable permet à PowerShell de valider la conformité des arguments saisis avant même le début de l'exécution :

- [string] : Chaîne de caractères (chemins, noms de serveurs, comptes).
- [int] : Nombre entier relatif (seuils, identifiants, compteurs).
- [switch] : Commutateur logique (drapeau vrai/faux activé par sa simple présence).

Exemple :

```
param(
```

```
[string]$DossierCible = "C:\Windows",  
[int]$SeuilTailleMo = 50,  
[switch]$ModeDetaille  
)
```

1.4 Valeurs par défaut

Affecter une valeur à une variable au sein du bloc param() définit sa valeur de repli (valeur par défaut). Si l'administrateur lance le script sans spécifier l'argument correspondant, PowerShell utilise cette valeur automatiquement. Le script demeure ainsi autonome tout en restant personnalisable.

1.5 Le paramètre de type commutateur ([switch])

Un paramètre de type [switch] ne prend pas d'argument textuel après son nom. Son comportement repose sur la présence ou l'absence du drapeau lors de l'appel :

- Si la commande est appelée sans mentionner le paramètre : la variable \$ModeDetaille s'évalue à \$false.
- Si la commande est appelée avec -ModeDetaille : la variable \$ModeDetaille s'évalue automatiquement à \$true.

2. Fonctions Avancées et Rôle de [CmdletBinding()]

2.1 Qu'est-ce que [CmdletBinding()] ?

L'attribut [CmdletBinding()] se place immédiatement au-dessus du bloc param(). Il indique au moteur PowerShell que le script ou la fonction doit être traité comme une véritable cmdlet système (fonction avancée).

```
[CmdletBinding()]  
param(  
    [string]$DossierCible = "C:\Windows"  
)
```

2.2 Avantages techniques majeurs

Activation de la validation stricte des arguments

Dans un script conventionnel sans [CmdletBinding()], si un utilisateur commet une faute de frappe sur un paramètre (par exemple -DossierCiblle "D:\Data"), PowerShell ignore l'erreur sans avertissement et stocke la valeur dans le tableau implicite \$args. Le paramètre réel \$DossierCible conserve alors sa valeur par défaut, entraînant un comportement inattendu.

Avec [CmdletBinding()], PowerShell bloque immédiatement le script et génère une erreur

explicite signalant qu'aucun paramètre correspondant n'a été trouvé.

Héritage des paramètres communs

L'ajout de `[CmdletBinding()]` injecte automatiquement les paramètres standards de PowerShell dans votre script, sans nécessiter la moindre ligne de code supplémentaire :

- `-Verbose` : Permet d'afficher des flux d'information détaillés via `Write-Verbose`.
- `-Debug` : Active le mode pas-à-pas via `Write-Debug`.
- `-ErrorAction` : Modifie le comportement en cas d'erreur pour l'ensemble du script.
- `-WhatIf` et `-Confirm` (si l'attribut `SupportsShouldProcess` est activé).

3. Conception et Bonnes Pratiques des Fonctions PowerShell

3.1 Objectif d'une fonction

Une fonction encapsule un bloc de logique réutilisable sous un identifiant unique. Elle répond au principe fondamental de génie logiciel "DRY" (*Don't Repeat Yourself*). Tout traitement répété plus d'une fois dans un script doit être extrait dans une fonction dédiée.

3.2 Règles de nommage : La nomenclature Verbe-Nom

PowerShell impose une convention standardisée : chaque fonction doit être nommée sous la forme Verbe-Nom au singulier.

- Le verbe doit impérativement provenir de la liste officielle des verbes approuvés par Microsoft (consultable en console via la commande `Get-Verb`).
- Exemples recommandés : `Write-LogMessage`, `Get-SystemHealth`, `Test-NetworkLink`.
- Formats proscrits : `logger()`, `ecrire_erreur()`, `DoAudit()`.

3.3 Déclaration des métadonnées de paramètres

Pour conférer de la robustesse à une fonction interne, ses paramètres peuvent recevoir des attributs de contrôle :

L'attribut `[Parameter(Mandatory = $true)]`

Rend obligatoire la transmission d'une valeur. Si l'appelant omet ce paramètre, PowerShell met l'exécution en pause et invite l'opérateur à saisir la donnée requise dans la console.

L'attribut de validation `[ValidateSet()]`

Restreint strictement les valeurs acceptées à une liste fermée de constantes. Tout passage d'une valeur non répertoriée déclenche un rejet immédiat avant l'entrée dans le corps de la fonction.

Exemple d'en-tête de fonction contrôlé :

```
function Write-LogMessage {
    param(
        [Parameter(Mandatory = $true)]
        [string]$Message,

        [Parameter(Mandatory = $true)]
        [ValidateSet("Information", "Avertissement", "Erreur")]
        [string]$Niveau
    )
    # Traitement
}
```

3.4 Portée des variables (Variable Scope)

Toute variable déclarée à l'intérieur d'une fonction appartient par défaut à sa portée locale (*Local Scope*). Elle n'écrase pas une variable de même nom présente dans le corps principal du script et cesse d'exister dès la fin de l'exécution de la fonction. Cette isolation garantit l'absence d'effets de bord indésirables.

4. Typologie des Erreurs : Non-Terminantes vs Terminantes

La compréhension du mécanisme de gestion d'erreur dans PowerShell repose sur une distinction architecturale majeure inexistante dans d'autres langages.

4.1 L'erreur non-terminante (Comportement par défaut)

Par défaut, la quasi-totalité des cmdlets natives génère des erreurs dites **non-terminantes** en cas d'échec opérationnel (par exemple : fichier introuvable, accès refusé à un répertoire).

- **Conséquence** : Le moteur PowerShell affiche le message d'erreur en rouge dans la console, mais continue immédiatement le déroulement du script à la ligne suivante.
- **Danger opérationnel** : Le script poursuit son traitement sur des données incomplètes ou inexistantes, risquant de propager l'incident sur les étapes ultérieures.

4.2 L'erreur terminante (Exception fatale)

Une erreur terminante interrompt immédiatement le flux d'instructions. C'est le seul type d'erreur capable d'activer un mécanisme de capture d'exception.

4.3 La règle d'or : Le rôle de -ErrorAction Stop

Un bloc de protection structuré (try / catch) est **totalemtent inopérant** face à une erreur non-terminante : PowerShell affichera l'erreur en console et ignorera complètement le bloc

catch.

Pour router un échec vers un bloc de traitement d'erreur, il est indispensable de forcer la commande à convertir ses erreurs non-terminantes en exceptions bloquantes via le paramètre :

-ErrorAction Stop

Exemple :

Erreur non-terminante : le catch est IGNORE

```
try {
    Get-ChildItem -Path "C:\Inexistant"
}
catch {
    Write-Host "Cette ligne ne sera jamais executee !"
}
```

Erreur terminante forcee : le catch INTERCEPTE l'incident

```
try {
    Get-ChildItem -Path "C:\Inexistant" -ErrorAction Stop
}
catch {
    Write-Host "L'erreur a bien ete interceptee."
}
```

5. Gestion Structurée des Exceptions : Le Bloc Try / Catch / Finally

5.1 Architecture globale

La structure de protection repose sur trois blocs interdépendants :

```
try {
    # 1. Sequence d'instructions susceptibles d'echouer
}
catch {
    # 2. Sequence corrective executee uniquement en cas d'incident terminant
}
finally {
    # 3. Sequence de cloture executee systematiquement
}
```

5.2 Fonctionnement détaillé de chaque segment

Le bloc try

Regroupe les opérations à risque (interrogation de chemins réseau, accès au système de fichiers, lecture de services critiques). Dès qu'une instruction échoue avec -ErrorAction Stop, le moteur quitte immédiatement le bloc try sans exécuter les lignes suivantes situées dans ce bloc.

Le bloc catch

Ce bloc ne s'exécute **que si une exception bloquante s'est produite** dans le try. C'est l'emplacement où l'administrateur implémente la journalisation de l'anomalie, l'envoi d'une notification ou l'application d'une stratégie de repli.

La variable automatique \$_ et l'objet exception

À l'intérieur du bloc catch, la variable automatique \$_ représente l'objet de l'erreur en cours de traitement.

- Pour extraire uniquement le message d'erreur clair renvoyé par le composant Windows sans la trace de pile technique, on interroge la propriété :
\$_Exception.Message

Le bloc finally

Le bloc finally est une garantie d'exécution. Qu'il y ait eu un déroulement sans accroc ou qu'une exception ait été levée, le code contenu dans ce bloc est exécuté sans exception.

- **Usage en production** : Fermeture de connexions réseau, libération de verrous sur des fichiers, réinitialisation de paramètres temporaires, consignation d'un statut final.

6. Aiguillage Multi-Critères : L'Instruction switch

6.1 Pourquoi remplacer les cascades de if / elseif / else ?

Lorsque l'on doit évaluer une variable selon plusieurs valeurs fixes (par exemple le niveau d'une alerte : Information, Avertissement ou Erreur), la syntaxe if / elseif devient verbeuse et alourdit la lisibilité :

```
if ($Niveau -eq "Information") { ... }  
elseif ($Niveau -eq "Avertissement") { ... }  
elseif ($Niveau -eq "Erreur") { ... }
```

6.2 Syntaxe et fonctionnement de switch

L'instruction switch teste directement la valeur transmise face à un ensemble de motifs prédéfinis :

```
switch ($Niveau) {  
    "Information" { Write-Host $Message -ForegroundColor Green }  
    "Avertissement" { Write-Host $Message -ForegroundColor Yellow }  
    "Erreur"      { Write-Host $Message -ForegroundColor Red }  
    Default       { Write-Host $Message -ForegroundColor White }  
}
```

- Chaque bloc d'action associé à la condition validée est exécuté.
- La clause facultative Default prend en charge toute valeur non répertoriée dans les branches précédentes.